



АСИНХРОННЫЙ КОД В C++

Анатолий Щеглов
ООО "МТС Веб Сервисы"



Содержание

- Синхронный код
- Коллбэки
- Корутины
- `std::execution`

```
struct socket_t;  
  
void sync_read(socket_t socket,  
               byte* buffer, size_t buffer_size);
```

- Читает в буфер.
- Возвращает управление только когда закончит чтение.

Введение - Асинхронный код

```
using callback_t = void(void* arg);

void async_read(socket_t socket,
                byte* buffer, size_t buffer_size,
                callback_t* callback, void* callback_arg);

for (;;) {
    run_completion_callbacks();
    wait_completions();
}
```

- Сразу возвращает управление.
- Читает в буфер в фоне.
- После завершения фонового чтения,
run_competion_callbacks() выполняет callback(callback_arg) .

Введение - История

- 2009 год - `boost::future<T>` , `.then()` .
- C++11 - блокирующий `std::future<T>::get()` без `.then()` .
- 2012 год - N3328 Resumable Functions.
- 2013-2015 годы - превью корутин в MS Visual C++ 2013 и 2015.
- 2018 - Coroutines TS (aka Гор-рутины, в честь Гора Нишанова).
- C++20 - корутины в стандарте.
- C++26? - `std::execution`

```
| len1 | data1 | len2 | data2 | len=0 |
|-----+-----|-----+-----|-----|
```

```

, -<-----<---.
'-> [read len] --> [read data] -> [process] -'
      \
      \ --> done if len==0

```

```
void Process(vector<byte> data);
```

M W S Синхронный код - API чтения

```
expected<size_t, error_code> ReadSome(int fd, span<byte> buf);
```

- Читает от 1 до buf.size()
- Возвращает размер прочитанного

Синхронный код - чтение всего буфера

```
expected<void, error_code> ReadAll(int fd, span<byte> buf) {  
    while (!buf.empty()) {  
        auto res = ReadSome(fd, buf);  
        if (!res) { return unexpected(res.error()); }  
        buf = buf.subspan(*res);  
    }  
    return {};  
}
```


Синхронный код - обработка соединения

```
expected<void, error_code> HandleConnection(int fd) {  
    for (;;) {  
        uint8_t data_size;  
        auto res = ReadAll(fd, as_writable_bytes(span{&data_size, 1}));  
        if (!res || data_size == 0) { return res; }  
        std::vector<byte> data(data_size);  
        res = ReadAll(fd, data);  
        if (!res) { return res; }  
        Process(std::move(data));  
    }  
}
```

Синхронный код - хвостовая рекурсия

```
expected<void, error_code> ReadAll(int fd, span<byte> buf) {  
    if (buf.empty()) { return {}; }  
    auto res = ReadSome(fd, buf);  
    if (!res) { return unexpected(res.error()); }  
    return ReadAll(fd, buf.subspan(*res));  
}
```

vs обычный цикл

```
expected<void, error_code> ReadAll(int fd, span<byte> buf) {  
    while (!buf.empty()) {  
        auto res = ReadSome(fd, buf);  
        if (!res) { return unexpected(res.error()); }  
        buf = buf.subspan(*res);  
    }  
    return {};  
}
```

CPS - continuation passing style

aka коллбэки

- Функции не возвращают результат через `return result;`
- Функции передают результат в коллбэк - `return callback(result);`

CPS - API чтения

```
void ReadSome(int fd, span<byte> buf,  
              function<void(expected<size_t, error_code>)> callback);
```

- Читает в `buf`.
- Вызывает `callback` с размером прочитанного.

CPS - чтение всего буфера - рекурсия

```
void ReadAll(int fd, span<byte> buf,  
             function<void(expected<void, error_code>)> callback) {  
    if (buf.empty()) { return callback({}); }  
    ReadSome(fd, buf, [=](expected<size_t, error_code> res) {  
        if (!res) { return callback(unexpected(res.error())); }  
        ReadAll(fd, buf.subspan(*res), callback);  
    });  
}
```

- Рекурсия вместо цикла.
- Неограниченная рекурсия если `ReadSome` вызывает коллбэк синхронно.

CPS - чтение всего буфера - аллокации

```
void ReadAll(int fd, span<byte> buf,  
             function<void(expected<void, error_code>)> callback) {  
    if (buf.empty()) { return callback({}); }  
    ReadSome(fd, buf, [=](expected<size_t, error_code> res) {  
        if (!res) { return callback(unexpected(res.error())); }  
        ReadAll(fd, buf.subspan(*res), callback);  
    });  
}
```

- Всё состояние в замыкании - `[=]` ,
- `function<> callback` не может поместиться в `function<>` без аллокации.

```
void HandleConnection(int fd,
                      function<void(expected<void, error_code>)> callback) {
    auto data_size = make_shared<uint8_t>();
    ReadAll(fd, as_writable_bytes(span{data_size.get(), 1})),
    [=](expected<void, error_code> res) {
        if (!res || *data_size == 0) { return callback(res); }
        auto data = make_shared<std::vector<byte>>(*data_size);
        ReadAll(fd, *data, [=](expected<void, error_code> res) {
            if (!res) { return callback(res); }
            Process(std::move(*data));
            HandleConnection(fd, callback);
        });
    });
}
```

- `data_size` и `data` должны иметь стабильный адрес на время чтения

CPS + struct - чтение всего буфера

```
struct Reader {  
    int fd;  
    function<void(expected<void, error_code>)> callback;  
    void Start(span<byte> buffer) {  
        buf = buffer;  
        if (buf.empty()) { return callback({}); }  
        ReadSome(fd, buf, [this](auto res) { OnRead(res); });  
    }  
    void OnRead(expected<size_t, error_code> res) {  
        if (!res) { return callback(unexpected(res.error())); }  
        Start(buf.subspan(*res));  
    }  
    span<byte> buf;  
};
```

- `[this]` - маленькое замыкание, без аллокаций в `function<>`

CPS + struct - обработка соединения

```
struct ConnectionHandler {  
    int fd;  
    function<void(expected<void, error_code>)> callback;  
    void Start() { size_reader.Start(as_writable_bytes(span{&data_size, 1})); }  
    void OnReadSize(expected<void, error_code> res) {  
        if (!res || data_size == 0) { return callback(res); }  
        data.resize(data_size);  
        data_reader.Start(data);  
    }  
    void OnReadData(expected<void, error_code> res) {  
        if (!res) { return callback(res); }  
        Process(std::move(data));  
        Start();  
    }  
    uint8_t data_size;  
    std::vector<byte> data;  
    Reader size_reader{fd, [this](auto res) { OnReadSize(res); }};  
    Reader data_reader{fd, [this](auto res) { OnReadData(res); }};  
};
```

CPS + синхронный возврат

```
optional<expected<size_t, error_code>> ReadSome(  
    int fd, span<byte> buf,  
    function<void(expected<size_t, error_code>)> callback);
```

```
void Start(span<byte> buffer) {  
    buf = buffer;  
    if (buf.empty()) { return callback({}); }  
    auto res = ReadSome(fd, buf, [this](auto res) { OnRead(res); });  
    if (res) { OnRead(*res); }  
}  
void OnRead(expected<size_t, error_code> res) {  
    if (!res) { return callback(unexpected(res.error())); }  
    Start(buf.subspan(*res));  
}
```

- Компилятор может распознать хвостовую рекурсию при `-O2`.

```
Start -> (inline) OnRead -> Start
```

Корутины

- Корутина - это функция с `co_return` или `co_await`.
- Корутина возвращает некоторый тип `Future`, у которого есть тип `Future::promise_type`.

```
struct Future { struct promise_type { ... }; };  
  
Future coro()      { co_return; }      // корутина  
Future not_coro() { return coro(); }    // не корутина
```

Корутины - API чтения

```
Future<expected<size_t, error_code>> ReadSome(int fd, span<byte> buf);
```

Корутины - чтение всего буфера

```
Future<expected<void, error_code>> ReadAll(int fd, span<byte> buf) {  
    while (!buf.empty()) {  
        auto res = co_await ReadSome(fd, buf);  
        if (!res) { co_return unexpected(res.error()); }  
        buf = buf.subspan(*res);  
    }  
    co_return {};  
}
```

vs синхронная версия:

```
expected<void, error_code> ReadAll(int fd, span<byte> buf) {  
    while (!buf.empty()) {  
        auto res = ReadSome(fd, buf);  
        if (!res) { return unexpected(res.error()); }  
        buf = buf.subspan(*res);  
    }  
    return {};  
}
```

Корутины - обработка соединения

```
Future<expected<void, error_code>> HandleConnection(int fd) {  
    for (;;) {  
        uint8_t data_size;  
        auto res = co_await ReadAll(fd, as_writable_bytes(span{&data_size, 1}));  
        if (!res || data_size == 0) { co_return res; }  
        std::vector<byte> data(data_size);  
        res = co_await ReadAll(fd, data);  
        if (!res) { co_return res; }  
        Process(std::move(data));  
    }  
}
```

Корутины - устройство

```
Future coro() {  
    // co_return co_await other();  
    struct CoroutineState {  
        Future::promise_type p_;  
        Future other_fut_;  
        int state_{0};  
        void resume() { switch (state_) {  
            case 0:  
                other_fut_ = other();  
                if (!other_fut_.await_ready()) {  
                    state_ = 1;  
                    return other_fut_.await_suspend(std::coroutine_handle{this});  
                }  
            case 1:  
                p_.return_value(other_fut_.await_resume());  
            }  
        }  
    };  
    auto* s = new CoroutineState{};  
    s->resume();  
    return s->p_.get_return_object();  
}
```

Корутины - свой код аллокации

```
Future coro(span<byte> mem) {  
    co_return 1;  
}  
  
struct Future {  
    struct promise_type {  
        template <typename... OtherArgs>  
        static void* operator new(size_t count, span<byte> mem, OtherArgs&&...) {  
            assert(count <= mem.size());  
            return mem.data();  
        }  
    };  
};
```

- Размер "CoroutineState" не `constexpr`, он становится известен только после всех оптимизаций.

Senders and Receivers

aka `std::execution`, C++26 [exec], [p2300].

- "Sender" - фабрика асинхронной операции.
- "Receiver" - принимает результат асинхронной операции, почти как коллбэк.
- Результат операции - это успех, ошибка, или остановка (отмена).

```
struct Receiver {  
    void set_value(V...);  
    void set_error(E);  
    void set_stopped();  
};
```

Senders and Receivers - пример

```
template <class Receiver> struct OperationState : NonMoveable {  
    Receiver r_;  
    void start() noexcept { std::move(r_).set_value(42); }  
};  
  
struct Sender {  
    using sender_concept = sender_t;  
    using completion_signatures = completion_signatures<set_value_t(int)>;  
    auto connect(receiver auto r) { return OperationState{.r_ = std::move(r)}; };  
};
```

```
sender auto s = Sender{};  
auto [result] = sync_wait(s).value();  
REQUIRE(result == 42);
```

Senders and Receivers - стандартные

- `just(value)` - сендер, который сразу отдает `value` .
- `then(sender_of<X>, [](X) -> Y {})` - добавляет коллбэк сендеру, "меняет" результат сендера.
- `let_value(sender_of<X>, [](X) -> sender_of<Y> {})` - возвращает новую асинхронную операцию, передавая в неё результат сендера.

```
int mul_2(int x) { return x * 2; }  
sender auto add_3(int x) { return just(x + 3); }  
  
sender auto snd = just(1) | then(mul_2) | let_value(add_3);
```

Senders and Receivers - any

```
any_sender ValueOrError(bool b) {  
    if (b) { return just(1); }  
    return just_error(2);  
}
```

- В функции два return с выражениями разных типов, но у функции может быть только один тип результата.
- Используем стирание типов.
- **Стирание типов может приводить к аллокации.**

Directed by
ROBERT B. WEIDE

Senders and Receivers - API чтения

```
struct ReadSome {
    template <class Receiver> struct OpState {
        int fd_;
        span<byte> buf_;
        Receiver r_;
        void start() noexcept {
            ReadSomeCPS(fd_, buf_, [this](auto res) { OnRead(res); });
        }
        void OnRead(expected<size_t, error_code> res) {
            auto&& r = std::move(r_);
            res ? r.set_value(*res) : r.set_error(res.error());
        }
    };
    int fd_;
    span<byte> buf_;
    using sender_concept = sender_t;
    using completion_signatures =
        completion_signatures<set_value_t(size_t), set_error_t(error_code)>;
    auto connect(receiver auto r) { return OpState{fd_, buf_, std::move(r)}; }
};
```

Senders and Receivers - чтение всего буфера

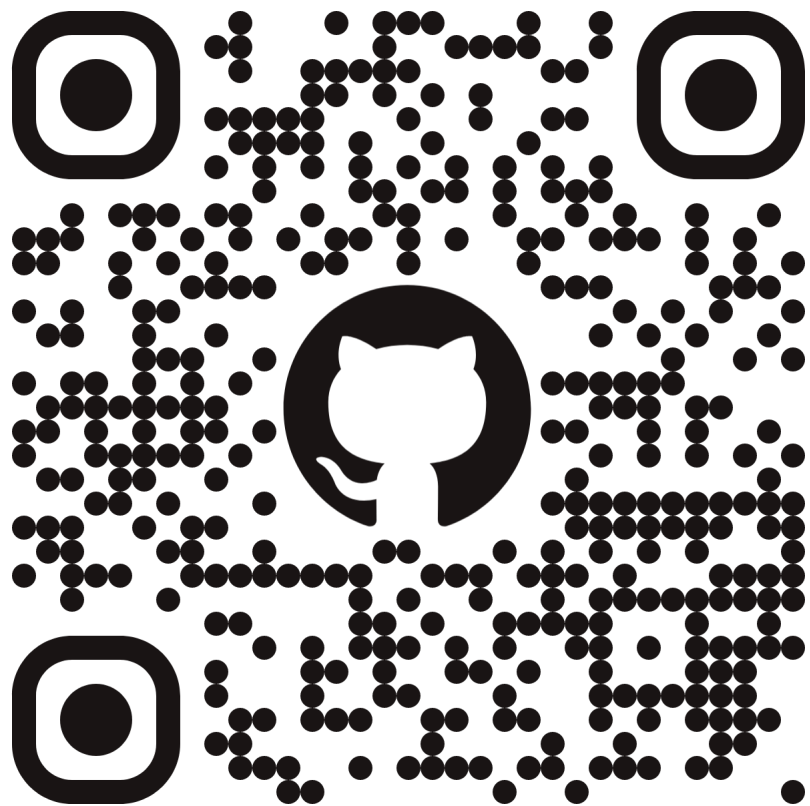
```
sender auto ReadAll(int fd, span<byte> buf) {  
    return just(buf) | let_value([fd](span<byte>& buf) {  
        return just() | let_value([&] {  
            return ReadSome{fd, buf} | then([&buf](size_t n) {  
                buf = buf.subspan(n);  
                return buf.empty();  
            });  
        });  
    }) |  
    repeat_effect_until();  
});  
}
```

Выводы

- Корутины хороши если аллокации не проблема.
- Коллбэки не требуют сильно больше кода.
- `std::execution` предлагает интересные идеи, (в т.ч. отдельные `set_value/set_error` вместо коллбэка), но в текущей реализации есть места для улучшений.

Вопросы?

Исходники кода на слайдах



<https://github.com/ascheglov/cxx-async-io/>

Чат про C++

